

Can MPNNs overcome short range oversquashing? - notes

- <https://docs.google.com/document/d/16R1E2ExKUCP5SIXWHr-KzbVDx9DBUclra-EbU8IB-iE/edit?tab=t.0> writing advice
- In generalised two-centre, we know that since central nodes are structurally identical, if they have the same initial node representations, an equivariant MPNN will evolve their representations identically.
- This is exactly what the Short range oversquashing paper discuss.
- To break the symmetry of the central nodes, the simplest modification (to either the graph or MPNN) we can do is give central nodes distinct, orthogonal, labels. (to avoid modifying the graph, we could also use rMPNN - Random Node Features)
- If the initial node labels of central nodes are different, theoretically, the network could learn to use them as separate lanes of communication. However, in practise (as seen in the short range oversquashing paper's results) SGD fails to find a parameter configuration which enables the MPNN to use the central nodes as separate lanes of communication.
- Why might this be the case?
- ▼ backpropagation reminder (also see handwritten notes)
 - Backpropagation works on a computational graph
 - Nodes are variables (tensors). These include your input, x , intermediate activations, h , weights, W , and your final loss, L
 - edges are mathematical operations (such as concatenation, matrix multiplication, addition, ReLU)
 - We compute the derivative of the final node (loss L), with respect to every other node in the computational graph.
 - For gradient descent, we of course care about the gradient of L with respect to the nodes which correspond to parameters.

- We 'backpropagate' gradients through the computational graph according to the chain rule:

$$\nabla_V L = \sum_{C \in \text{children}(V)} (\nabla_C L) \frac{\partial C}{\partial V}$$

- Intuitively, the influence of node V on the final loss L, is equal to the sum of influences of each child of V on the final loss (weighted by the influence of node V on that child).
- this is precisely the chain rule shown above.
-
- remember that in a single MPNN layer, ψ , is fixed.
- ψ is implemented as an MLP usually.

$$\mathbf{h}_u = \varphi \left(\mathbf{x}_u, \bigoplus_{v \in \mathcal{N}_u} \psi(\mathbf{x}_u, \mathbf{x}_v) \right). \quad (5.56)$$

- In the computational graph, the weights of ψ , is connected to $\mathbf{h}_c^1$ (i.e. the representation of the central node, after the first MPNN layer).
- But since the central node is connected to ALL target nodes, $\mathbf{h}_c^1$ is connected (indirectly) to $\mathbf{h}_{t_i}^2$ for all target nodes, t_i .
- Therefore the gradient of all $\mathbf{h}_{t_i}^2$ with respect to loss L, is propagated back to the gradient of $\mathbf{h}_c^1$ (and hence $\mathbf{W}_{\psi_1}$)
- Therefore, during gradient descent, $\mathbf{W}_{\psi_1}$ is encouraged to update in a direction which enables $\mathbf{h}_{t_i}^2$ (for all i) to better match with their true labels.
- $\mathbf{W}_{\psi_1}$ cannot, for example, specialise in routing information from a single source to the corresponding target, and let future MPNN layers route information from the remaining sources to targets.

- Instead, due to the fact that all targets are connected to the centre node, in 2-radius, W_{ψ_1} is forced to route information to the target nodes all at once.
- This requires W_{ψ_1} to map to a large dimension to store all the required information, which is precisely the bottleneck effect.
- Note that even in generalised k-centre, since all central nodes are connected to the target nodes, we will still have the same problem.
- This time, in the computational graph, W_{ψ_1} is connected to $h_{c_j}^1$ for all central nodes, c_j , and each $h_{c_j}^1$ will still be connected to all of $h_{t_i}^2$
- And so the gradient of ALL of $h_{t_i}^2$ gets propagated back to W_{ψ_1} .
- Even if we let all central nodes have their own message function, ψ_j (either by breaking permutation equivariance of the MPNN layer, or by using a custom implementation of virtual nodes), $W_{\psi_{1j}}$ would still be connected to all of $h_{t_i}^2$ in the computational graph.
- And so really to overcome the bottleneck effect, we have to stop letting gradients flow from ALL of the target nodes to a central node for an MPNN layer.
- This motivates several ideas

▼ dropedge

- simply, if h_c^1 is receiving n gradients from n different targets (with each gradient conflicting with one another), randomly removing these gradients will reduce the gradient interference
- W_{ψ_1} only has to adjust itself to a small mini batch of targets at a time.
- Furthermore, if you have several central nodes, with those central nodes now being connected to different targets (after dropping edges), then those central nodes can now be used as separate lanes of communication (rather than points of communication to ALL nodes).
- This will effectively increase the representational capacity of the set of central nodes.

However, there is a problem:

- in order to pass information from target B to corresponding source A
- both edges from A to centre, and B to centre must survive for at least one central node.
- probability of it surviving for one central node is p^2 (where p is 1- drop rate)
- and so for high drop rate, both required edges are unlikely to survive
- Having more central nodes does increase the probability of both required edges surviving for at least once central node, however, this is not a perfect solution.
- this motivates letting the network learn which edges to drop rather than dropping them randomly.
- another motivation for this is that two-centre is a routing problem. and so we need some mechanism by which the network can learn how to route information between the correct nodes. Transformers learn this routing mechanism via key-query lookups.

Another question is, do you keep droppedge on at test time?

- if you train with droppedge, then this is an inductive bias which encourages the model to use central nodes as seperate lanes of communication.
- and so as long as you scale accordingly at test time (keeping droppedge off), it should still continue to use central nodes as seperate lanes of communication.
- keeping droppedge on might mean that in a single message passing step, the edges between corresponding source and target that connect to the central node may be dropped
- but if you increase the number of message passing layers, there should be one layer in which neither of those edges are dropped.
- actually both of these fail...
- turning droppedge off at test time means ψ (when computing target node features), will recieve superposition of ALL source nodes. it will not know

how to handle this, it is only used to disentangling superposition of a few source nodes.

- keeping dropout on at test is more promising. but at each layer, messages from each node spread and smooths, and so it becomes quite hard to pass specific messages if you increase the number of layers too much.

Either way, we need some other way of fixing the issue of the weight matrix needing to address all targets at once

- this motivates the following approaches.

▼ fix A

Instead of *randomly* dropping edges, what if we let the network *learn* which edges to drop

- In a standard MPNN, the update rule for a target node t indiscriminately adds up every central neighbor's transformed feature. There is an implicit weight of 1 on every edge:

$$\mathbf{h}_t^{(2)} = \phi \left(\mathbf{h}_t^{(1)}, \sum_{c \in \mathcal{C}} \mathbf{1} \cdot \psi(\mathbf{h}_t^{(1)}, \mathbf{h}_c^{(1)}) \right)$$

- (Because every edge is weighted equally, this also means that the weight matrix for ψ is forced to project N different source node representations into N perfectly orthogonal subspaces so they don't destructively interfere during the summation. This is very difficult!)
- So instead, we can compute a learnable scalar $\alpha_{t,c}$ for each edge $t \rightarrow c$. This is computed using $\mathbf{h}_c$ and $\mathbf{h}_t$.
- $\alpha_{t,c}$ should describe, 'given my features $\mathbf{h}_t$, and your features $\mathbf{h}_c$, how much do I need you?'
- furthermore, $\alpha_{t,c}$ is softmax normalised (across all neighbours of t , which will all be central nodes in the case of 2-radius):

$$\alpha_{tc} = \text{softmax}_c(e_{tc}) = \frac{\exp(e_{tc})}{\sum_{k \in C} \exp(e_{tk})}$$

- Because of the softmax, the sum of all $\alpha_{t,c}$ for a given target must equal 1. And so If the network learns its job correctly, the attention mechanism acts as a router. It will learn to ignore some central nodes, and pay attention to others.
- Especially if the central nodes are not fully connected to all sources and targets, the targets will benefit from paying attention to only the central nodes which are connected to the corresponding source node of that target.
- But even still if all central nodes are fully connected, the target nodes may still learn to pay attention to only some of the central nodes.

We have just described the attentional flavour of GNNs (distinct to graph transformers)

In the **attentional** flavour (Veličković et al. 2018; Monti et al. 2017; J. Zhang et al. 2018; Brody, Alon, and Yahav 2022), these interactions are implicit:

$$\mathbf{h}_u = \varphi \left(\mathbf{x}_u, \bigoplus_{v \in \mathcal{N}_u} a(\mathbf{x}_u, \mathbf{x}_v) \psi(\mathbf{x}_v) \right). \quad (5.55)$$

- in the attentional flavour, nodes can still only communicate with its neighbours.
- in graph transformers, nodes can attend to all nodes in the graph (global receptive field), but attention is influenced by graph structure. This is still $O(N^2)$
- Graphormer for example still has an $N \times N$ attention matrix.

How is attention implemented?

- GAT and TransformerConv are two main methods.

▼ GAT

1. The Shared Linear Projection:

First, every single node in the graph (both targets and central nodes) gets multiplied by the *same* learnable weight matrix $\mathbf{W}$. This transforms the raw input features into a new "hidden" space.

- Target projected: $\mathbf{z}_t = \mathbf{W}\mathbf{h}_t$
- Central projected: $\mathbf{z}_c = \mathbf{W}\mathbf{h}_c$

2. The Attention Scoring (The "Router"):

To figure out how much Target t should care about Central c , GAT concatenates ($\parallel$) their projected features and multiplies them by a learnable vector $\mathbf{a}$ (which acts as the attention neural network):

$$e_{tc} = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{z}_t \parallel \mathbf{z}_c])$$

Notice that it uses $\mathbf{z}_t$ and $\mathbf{z}_c$ (which were created using $\mathbf{W}$).

3. The Softmax:

The raw scores are normalized:

$$\alpha_{tc} = \frac{\exp(e_{tc})}{\sum_{k \in C} \exp(e_{tk})}$$

4. The Aggregation (The "Payload"):

Finally, Target t aggregates the features from the central nodes, weighted by α .

$$\mathbf{h}'_t = \sigma \left(\sum_{c \in C} \alpha_{tc} \mathbf{z}_c \right)$$

(Notice that $\mathbf{z}_c$ is just $\mathbf{W}\mathbf{h}_c$. So yes, it is the exact same $\mathbf{W}$!)

- importantly, the weight matrix $\mathbf{W}$ is shared between doing 2 jobs.
- It must transform the features so that $\mathbf{a}^\top$ can figure out if a node should attend to its neighbour.
- $\mathbf{W}^\top$ must also transform the features, $\mathbf{h}_c$, of neighbouring central nodes in a useful way so that those transformed features can usefully update the target node, $\mathbf{h}_t$.
- this motivates separating $\mathbf{W}$ into 3 separate matrices for $\mathbf{K}$, $\mathbf{Q}$ and $\mathbf{V}$:

1. Separate Projections (Query, Key, Value):

- **Query (What I want):** The target node computes its query: $\mathbf{q}_t = \mathbf{W}_Q \mathbf{h}_t$
- **Key (What I have):** The central node computes its key: $\mathbf{k}_c = \mathbf{W}_K \mathbf{h}_c$
- **Value (The actual payload):** The central node computes its value: $\mathbf{v}_c = \mathbf{W}_V \mathbf{h}_c$

2. The Attention Scoring (Dot Product):

Instead of concatenating and using a neural network vector $\mathbf{a}$, Transformers simply take the dot product of the Query and the Key.

$$e_{tc} = \frac{\mathbf{q}_t \cdot \mathbf{k}_c}{\sqrt{d}}$$

3. The Softmax:

$$\alpha_{tc} = \text{softmax}_c(e_{tc})$$

4. The Aggregation:

The target node aggregates the **Values** (not the Keys!):

$$\mathbf{h}'_t = \sum_{c \in C} \alpha_{tc} \mathbf{v}_c$$

- this is what TransformerConv does

▼ TransformerConv

- this is popularised by popularized by Shi et al., 2020
- in Q-K-V Transformer, $\mathbf{W}_Q$ and $\mathbf{W}_K$ learn exclusively to process the identifiers. $\mathbf{W}_V$ can learn exclusively to process the labels. This is a nice inductive bias for routing.
- Compare this to GAT, where a single matrix $\mathbf{W}$ must be optimised to do all 3 jobs at once.
- For TransformerConv the optimisation landscape is easier to navigate.

By restricting the attention to local neighbors, the computational complexity drops from $O(N^2)$ to $O(|E|)$, where $|E|$ is the number of edges. Because most real-world graphs are highly sparse ($|E| \ll N^2$), this is incredibly fast.

▼ fix B

- In a standard MPNN, every node must broadcast to all its neighbors, and every node must listen to all its neighbors.
- In the two-radius problem, this forces the central node to listen to all N sources and broadcast its compressed feature vector to all N targets simultaneously
- Instead, Co_GNN dynamically chooses how each node interacts with its neighbours at each layer.
- A node can choose one of 4 actions:
- STANDARD (S): Broadcast to all neighbors, and listen to all neighbors.
- LISTEN (L): Listen to all neighbors, but do not broadcast to any neighbours.
- BROADCAST (B): Broadcast to all neighbors, but do not listen to any neighbours.
- ISOLATE (I): Neither listen nor broadcast.
- To decide which action a node takes, Co-GNN introduces an Action network.
- A layer, π , of an action network has the following form

$$\mathbf{p}_i^{(t)} = \pi \left(\mathbf{h}_i^{(t)}, \bigoplus_{j \in \mathcal{N}_i} \mathbf{h}_j^{(t)} \right)$$

- for each node, i , π must output a probability distribution over the 4 actions, using the representation of node i (at the corresponding layer), alongside the representations of the neighbouring nodes.
- The node then samples an action $a_i^t \sim p_i^t$
- to allow backpropagation through this discrete sampling, the authors use the Gumbel-Softmax estimator

- Now let's define two binary indicator variables based on the chosen action a_i^t

- $Receive_i \in \{0, 1\}$: Is 1 if $a_i \in \{\text{Listen, Standard}\}$, else 0.
- $Send_j \in \{0, 1\}$: Is 1 if $a_j \in \{\text{Broadcast, Standard}\}$, else 0.

- Now the message passing equation becomes as follows:

$$\mathbf{h}_i^{(t+1)} = \phi \left(\mathbf{h}_i^{(t)}, \bigoplus_{j \in \mathcal{N}_i} \underbrace{(Receive_i^{(t)} \cdot Send_j^{(t)})}_{\text{The Learned Gate}} \cdot \psi(\mathbf{h}_i^{(t)}, \mathbf{h}_j^{(t)}) \right)$$

- A message only flows from node j to node i if node j is sending AND node i is receiving

how does Co-GNN solve the gradient interference problem?

- Remember we want to update the weight matrix W for the message function ψ . To do this, backprop must compute the gradient of the loss L with respect to the central node's representation $\mathbf{h}_c$
- In a standard MPNN, the gradient is roughly

$$\nabla_{\mathbf{h}_c} L = \sum_{t \in \text{Targets}} (\nabla_{\mathbf{h}_t} L) \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_c} \approx \sum_{t \in \text{Targets}} (\nabla_{\mathbf{h}_t} L) W$$

- here, we have gradients $\nabla_{\{\mathbf{h}_t\}} L$ from all target nodes t being backpropagated. this is gradient interference, which makes it difficult to update W .
- However for Co-GNN,

$$\nabla_{\mathbf{h}_c} L \approx \sum_{t \in \text{Targets}} (Receive_t \cdot Send_c) \cdot (\nabla_{\mathbf{h}_t} L) W$$

- the central node may not necessarily receive gradients from every target.
- hopefully during gradient descent, the network will learn that forcing the central node to talk to all targets yields a bad loss. and so it instead learns a cooperative routing strategy.
- for example, the Action Network might learn to set the central node to broadcast, but targets 2 through N to isolate, while target 1 to listen. and this way the central node can send a message specifically to target node 1.

How does Co-GNN compare to attentional GNN?

- **GAT (Attention)** solves gradient interference using a *soft, continuous gate* a_{ij} in $[0,1]$
- **Co-GNN** solves gradient interference using a *hard, discrete gate*: $\text{Receive}_i * \text{Send}_k$ in $\{0,1\}$
- note that in Co-GNN, there is less granularity than in GAT (or attention GNN). In GAT, we choose gate a_{ij} for every single ordered pair of nodes.
- but in Co-GNN, we make a choice for every node. and the node-level choices determine the gate for every edge.
- a difficulty of Co-GNN is that you can't backpropagate through a hard argmax choice. training requires careful tuning of a "temperature" hyperparameter. It is notoriously trickier to optimise compared to attention.

We can combine Co-GNN and GAT

- (and this is what the CO-GNN architecture already does I think)

Because Co-GNN is a framework, it consists of two networks:

1. **Action Network (π):** Decides $\{S, L, B, I\}$.
2. **Environment Network (η):** The actual GNN that passes the messages along the surviving edges.

- the action network is a standard GNN / MPNN
- but the environment network is an Attentional GNN

- so the Action network drops irrelevant nodes entirely.
 - On the *surviving* edges, the GAT environment network applies edge-level attention. It does fine-grained routing that the action network can't do.
- ▼ fix C: try several virtual nodes on GCN.
- 'the graph is divided into clusters (either randomly or using a clustering algorithm like METIS). Each real node is assigned to exactly *one* virtual node. A real node only receives the embedding from its explicitly assigned virtual node'
 - <https://gemini.google.com/app/136da927aea76828>

▼ *Exphormer: Sparse Transformers for Graphs*

- We augment the original graph to create an Interaction Graph, H
- To our original graph, we add global virtual nodes, which are connected to all original nodes.
- We also add a d -regular expander graph on top of it (where d is a hyperparameter)
- every node will get d new random edges.
- to choose which random edges, we generate d random permutations of our original N nodes.
- e.g. in permutation 1, if node 2 gets mapped to node 4, we add an edge from node 2 to 4 . this is an expander edge.
- now once the interaction graph H is build, exphormer runs an attentional GNN on it.

$$\mathbf{h}_i^{(t+1)} = \sum_{j \in \mathcal{N}_H(i)} \alpha_{ij} (\mathbf{W}_V \mathbf{h}_j^{(t)})$$

The neighbors $\mathcal{N}_H(i)$ are all nodes connected to i in that combined interaction graph.

- this is essentially just TransformerConv, however, there is one technical detail.

- Edges in the interaction graph H can be classified as local (edge from the original graph, expander edge, or global edge (i.e. connected to one of the global virtual nodes)).
- ▼ the attention mechanism $a(h_i, h_j)$ is conditioned on these edge features.
 - basically, for each edge type, we map to a learnable embedding / bias
 - we just add this bias to the QK dot product, before the softmax is applied.
 - conditioning attention on edge type gives more control over information flow. how much you attend to a node can depend on the type of edge to that node (whether it is a local edge, expander edge, or global edge)

Step 1: The Standard Attention Score (Semantic Similarity)

First, the model computes the standard Q-K dot-product attention score. This score only depends on the node features and asks, "How semantically similar is node i to node j ?"

$$e_{ij}^{\text{nodes}} = \frac{(\mathbf{W}_Q \mathbf{h}_i) \cdot (\mathbf{W}_K \mathbf{h}_j)}{\sqrt{d}}$$

Step 2: The Edge Feature and the Learnable Bias (Structural Priority)

Now, we need to inject the *type* of the edge into this calculation. Expormer does this by creating a learnable "bias" for each type of edge.

1. **Assign Edge Features:** Every edge in the interaction graph H is given a categorical feature. For example:

- Edge (i, j) from the original graph gets the feature `is_local`.
- Edge (i, j) from the expander gets the feature `is_expander`.
- Edge (i, v) connecting to a virtual node gets the feature `is_global`.

2. **Create a Lookup Table:** The model creates a small, learnable lookup table (an embedding layer) that maps each of these categorical features to a single scalar number.

- `is_local` $\rightarrow b_{\text{local}}$
- `is_expander` $\rightarrow b_{\text{expander}}$
- `is_global` $\rightarrow b_{\text{global}}$

These b values are learnable parameters, just like the weights in $\mathbf{W}$.

Step 3: The Final Conditioned Attention Score

The final attention score is simply the sum of the semantic score and the structural bias:

$$e_{ij}^{\text{final}} = e_{ij}^{\text{nodes}} + b_{\text{type}(i,j)}$$

where `type(i, j)` is just a function that returns the type of edge connecting i and j .

Why is expormer helpful for 2-radius?

- In the Two-Radius problem, the central node receives a sum of conflicting gradients from all N targets.
- In Expormer, because there are so many different paths, the gradient of the loss doesn't all accumulate at a single node
- the error signal from target 1 may backpropagate to a source node, for example.
- It would be amazing if we managed to connect a target node directly to its corresponding source node
- that will unlikely happen.
- However, there is an opportunity for a source to pass its label to the corresponding target via an alternative path (which will still likely go through the centre node).
- this puts less pressure on the central node in the first message passing step. And so the weight matrix of the message function in the first layer, is not being pulled in N different directions at once
- the global virtual node is not very helpful since it creates a bottleneck.
- the core idea from this paper, therefore, is the expander graph. (and combining it with attention)
- this is an alternative to local virtual nodes.
- are local virtual nodes (which are connected to a subset of the original nodes) or an expander graph, more useful for the 2-radius problem?

why local virtual nodes may be better than expander edges

- with K LVNs, you essentially create K mini 2-radius problems. (almost since you still have the original central node)
- The gradient from e.g. Target 1 physically cannot reach e.g. LVN 2.
- Gradient interference is **completely eliminated by design**
- However, for an expander graph, you are creating so many new routes through the network.
- You might create a 'nice' path which is straight from source to corresponding target.
- but you might also create very messy paths, from e.g. sources to sources, or longer paths from source to target back to central node, then to a different source etc...
- K mini 2-radius problems seems like a better inductive bias for the 2-radius problem.
- But note that both expander graph and virtual nodes suffer from picking subsets which just don't include a target and corresponding source.

▼ *Understanding Virtual Nodes: Oversquashing and Node Heterogeneity*

- here, we still have global virtual nodes.
- the trick is, for layer $l+1$, to perform message passing first, then perform the global node aggregation, before broadcasting the global node's message to all other nodes.

1. **Phase 1: Local Message Passing.** Every node i first computes an *intermediate* representation, $\mathbf{h}_{i,loc}^{(l+1)}$, by aggregating messages **only from its direct neighbors in the original graph**. This is a standard MPNN step.

$$\mathbf{h}_{i,loc}^{(l+1)} = \text{up}_{loc}^{(l)} \left(\mathbf{h}_i^{(l)}, \text{agg}_{loc}^{(l)}(\{\mathbf{h}_j^{(l)} : j \in \mathcal{N}_i\}) \right)$$

2. **Phase 2: Global Aggregation.** The Virtual Node (VN) now performs its aggregation. Crucially, it does **not** aggregate the old features $\mathbf{h}_j^{(l)}$. It aggregates the new, locally-updated features $\mathbf{h}_{j,loc}^{(l+1)}$ from Phase 1.

$$\mathbf{h}_{vn}^{(l+1)} = \text{up}_{vn}^{(l)} \left(\mathbf{h}_{vn}^{(l)}, \text{agg}_{vn}^{(l)}(\{\mathbf{h}_{j,loc}^{(l+1)} : j \in V\}) \right)$$

3. **Phase 3: Final Update / Broadcast.** The final output for node i , $\mathbf{h}_i^{(l+1)}$, is computed by combining its *local* update from Phase 1 with the *global* update from the VN in Phase 2.

- and this helps route messages through the global virtual node better somehow, rather than the central virtual node passing a message which just mashes together information from all nodes
- the mechanism for this is a bit tricky....

▼ *Probabilistic Graph Rewiring via Virtual Nodes*

- say we want v virtual nodes. and we want each node to be connected to d of those virtual nodes.
- so essentially, we run an MPNN layer (or a few).
- Then we compute v logits for each node by passing the representation of each node through an MLP. The logits represent the probability of that node being connected to each of the virtual nodes)
- we sample d times from those logits, this gives us the d connections of each original node to the virtual nodes.
- then we just run more MPNN layers on this updated graph (which now has virtual nodes).

can this be useful for two-radius?

- lets say we apply MLP to initial node representation (before message passing) to get logits.
- MLP may learn 'if it has identifier 1 map to virtual node 1', it would be able to do this for both source AND target.
- and so there is a high chance corresponding source and target get mapped to the same virtual node.
- and then we can just run attentional GNN on this.
- attentional GNN is good because each virtual node still has d connections, so we want to be able to drop edges if we can.

▼ does the idea really work in theory?

- each source and target will have its identifier in its representation.
- an MLP could learn to ignore label and just focus on identifier.

- MLP would then give high probability to the same virtual nodes for both source and target.
 - and this already makes two-radius much easier
- ▼ is it computationally efficient?
- you run an MLP once on each of the n nodes.
 - then it is just standard message passing.
 - seems ok to me.
- The MLP may find it hard to map corresponding source and targets to the same virtual node, or it may also ending up mapping nodes to only a small subset of the virtual nodes...